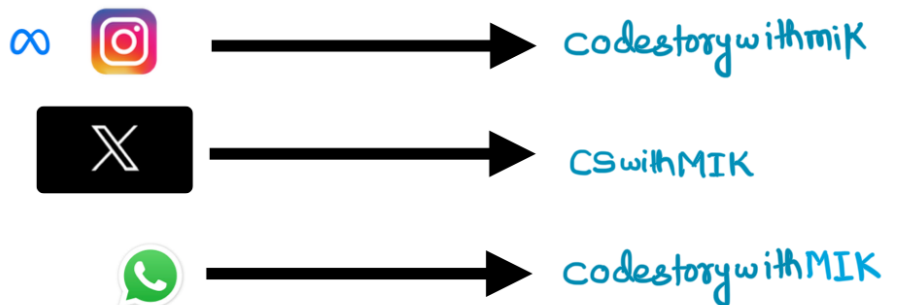
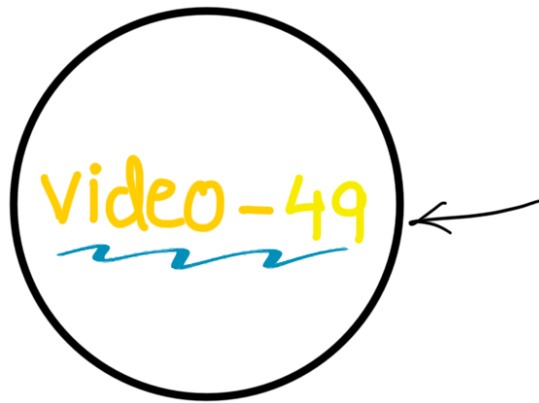
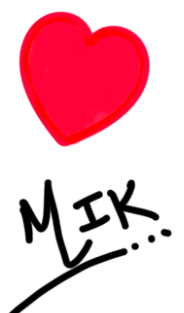


Graph Concepts & Qns



Motivation:-

Stop counting the hours left in the day and start making every single hour count



0-1 BFS

WHAT? WHY? HOW?

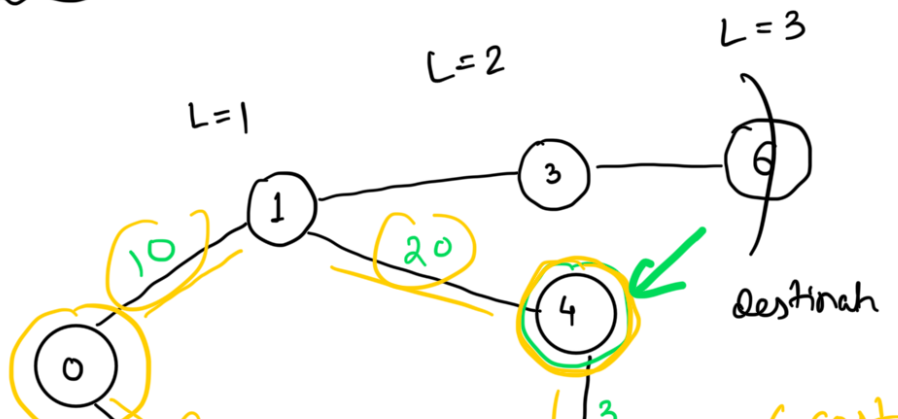


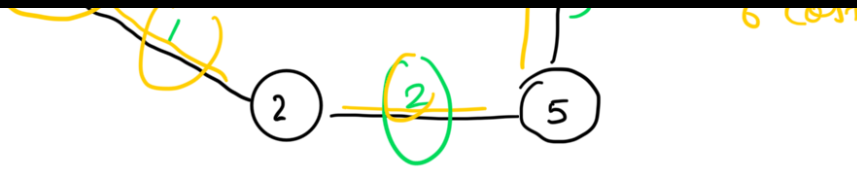
- Variation of simple BFS.
- Used to find single source shortest path to all nodes in a weighted graph.
- All edges have weight → 0 or 1

0-1 BFS → ???

Dijkstra → $O(E * \log V)$

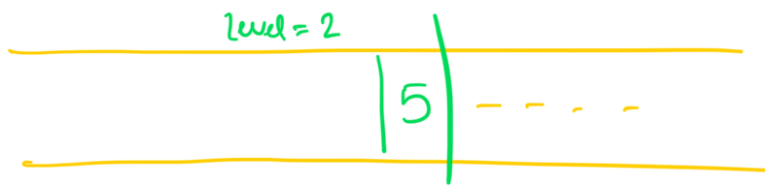
Let's first look at the standard simple
Khandani template BFS. (level order Tr).





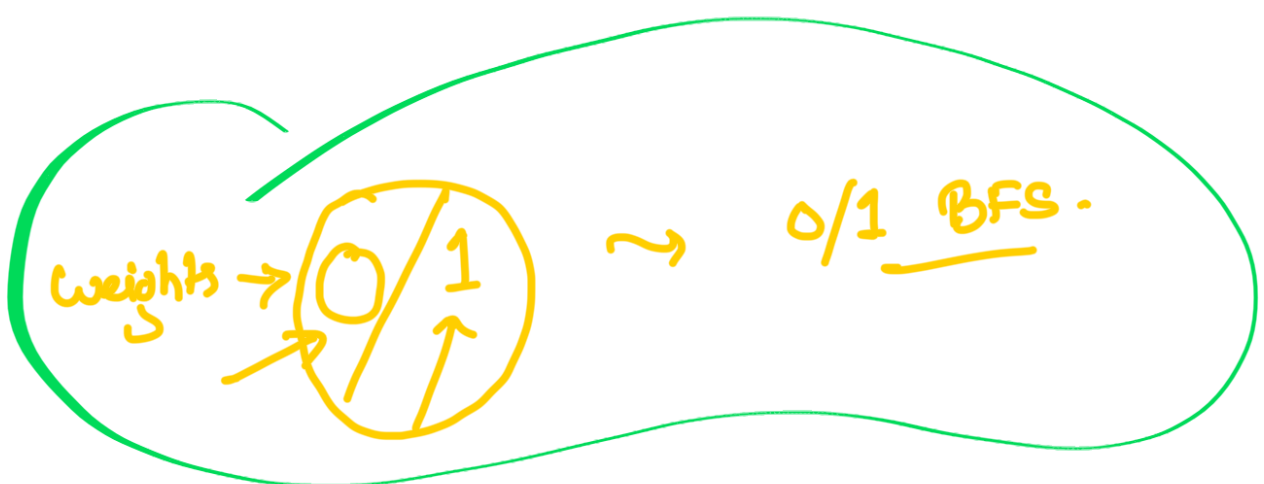
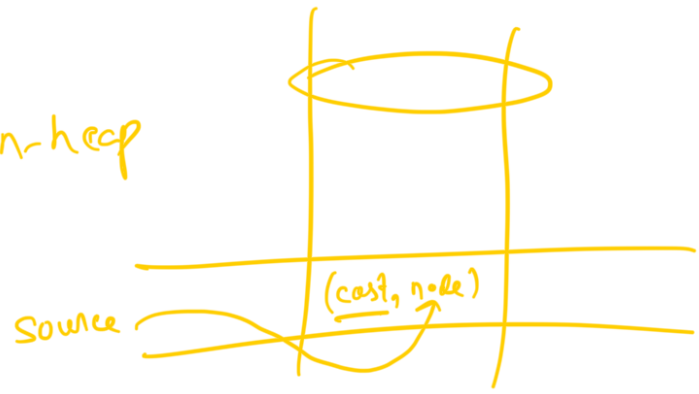
level = 2

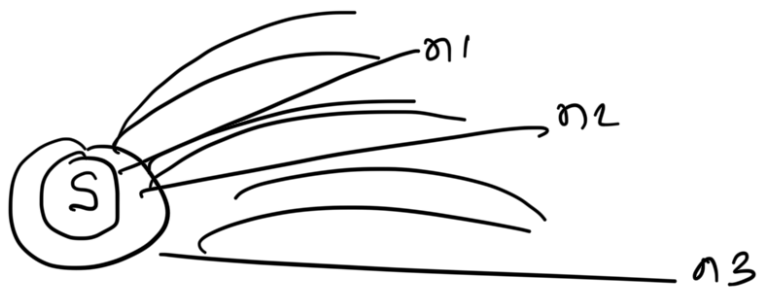
min-heap



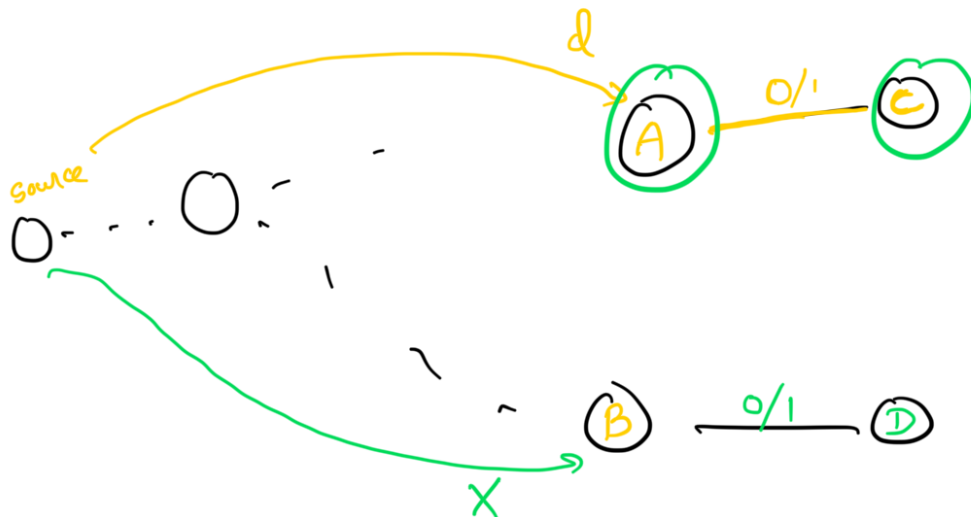
- Queue $\rightarrow$ At max two levels
- BFS $\rightarrow$ Each edge has same weight.

min-heap





Dijkstra's.

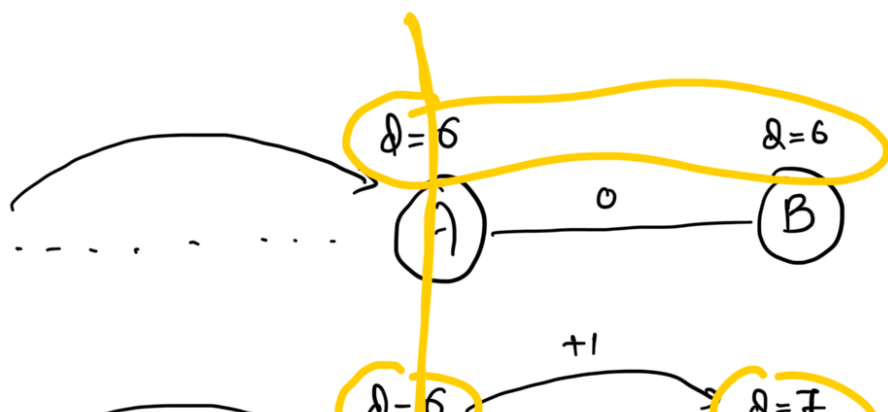


$$d+0 = d$$

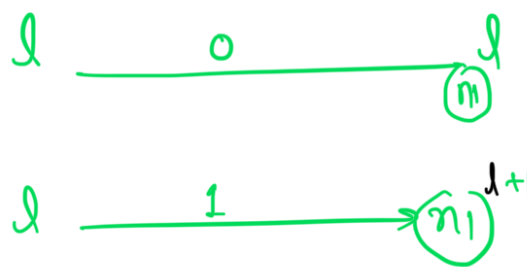
$$d+1 = d+1$$

$$x+0 = x$$

$$x+1 = x+1$$



normal
BFS

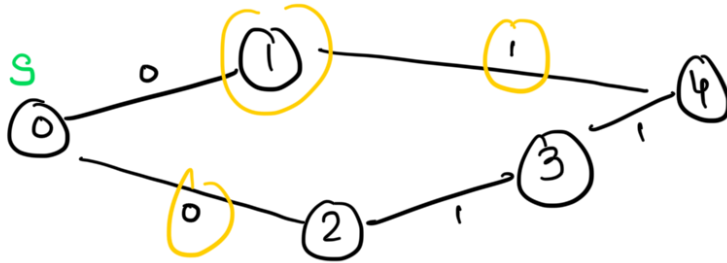


Deque.



0-1 BFS $\Rightarrow$ Deque

1 Day Run. (OI BFS)



result

0	1	2	3	4
0	0	0	1	1

cost = 1
node = 4

deque

$$1 + 1 = 2$$

```
if (edge == 0)
    front();
else
    back();
```

Code 0-1 BFS



Same as Dijkstra
(without Heap)
use: Deque

Dijkstra's...

```
class Solution
{
public:
    //Function to find the shortest distance of all the vertices
    //from the source vertex S.
    vector<int> dijkstra(int V, vector<vector<int>>> adj[], int S) {
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
        vector<int> result(V, INT_MAX);

        result[S] = 0;
        pq.push({0, S});
        //NOTE - You can add a visited vector to avoid revisiting a node again and again. It will
        while(!pq.empty()) {
            int d = pq.top().first;
            int node = pq.top().second;
            pq.pop();

            for(auto &vec : adj[node]) {
                int adjNode = vec[0];
                int dist = vec[1];

                if (d + dist < result[adjNode]) {
                    result[adjNode] = d + dist;
                    pq.push({d+dist, adjNode});
                }
            }
        }

        return result;
    }
};
```

0-1 BFS ...

```
(*) deque <pair<int, int>> dq;  
(*) result  
(*) result[s] = 0;  
(*) dq.push-front({0, 0});  
(*) while (!dq.empty()) {  
    [cost, node] = dq.pop-front();
```

```
    for (nbr: node->neighbors) {
```

```
        if (cost + wt < result[nbr]) {
```

```
            result[nbr] = cost + wt;
```

```
            if (wt == 0)  
                push-front();
```

```
            else
```

```
                push-back();
```

sorted
queue

Leetcode: 3286.

(0-1 BFS)
([k]n) →